

Let the Agent Do It? How Software Practitioners Understand and Make Permission Decisions in Agentic AI Assistants

LARISSA SALERNO, Chalmers University of Technology and University of Gothenburg, Sweden

HAOYU GAO, The University of Melbourne, Australia

GREGORY GAY, Chalmers University of Technology and University of Gothenburg, Sweden

ALEXANDER SEREBRENİK, Eindhoven University of Technology, The Netherlands

PHILIPP LEITNER, Chalmers University of Technology and University of Gothenburg, Sweden

Agentic AI assistants increasingly act on developers' behalf by modifying files, executing commands, and accessing external resources. These actions often require permission, yet little is known about how practitioners make permission decisions while still benefiting from agent autonomy. To address this gap, we conducted a sequential mixed methods study, interviewing 18 practitioners who use AI agents and then surveying 115 practitioners based on the interview findings. We find that practitioners often understand agent behaviour through what they can directly observe and review, while decisions, data use, and other activity behind the scenes remain less clear. This uncertainty also shapes permission decisions, which depend on the scope and risk of an action, whether it fits the task, familiarity with the agent, and the environment in which it operates. Practitioners respond by adjusting how closely they oversee agents, from setting limits in advance to monitoring execution and reviewing work afterwards. How much scrutiny they apply depends on factors such as trust, task importance, time pressure, and the consequences of an action. Our findings suggest that permission systems should make consequential actions easier to review, distinguish what an agent is allowed to do from what the user intended, make reversibility clearer, avoid treating repeated approvals as stable preferences, and distinguish rejecting a single action from rejecting an entire approach.

CCS Concepts: • **Software and its engineering** → **Software development process management**; *Software notations and tools*; • **Security and privacy** → *Access control*.

Additional Key Words and Phrases: Agentic AI, Coding Agents, AI-assisted Software Development, Permission Decisions, Permission Systems, Developer Oversight

1 Introduction

AI agents are being applied across software development workflows, including requirements analysis [43], code generation [60], code review [36], and test generation [4]. This introduces a development paradigm where software practitioners communicate their goals through natural language and delegate the selection and execution of concrete actions to agents. Such delegation creates a need to manage permissions for actions chosen by the agent as a task unfolds. *Runtime permission requests* ask software practitioners whether an agent may perform operations such as executing commands, modifying files, or accessing external resources, providing opportunities to control how delegated work proceeds [39].

Prior work has begun to explore permission mechanisms for AI agents and the challenges these tools introduce, from limited transparency in commercial approval workflows to the difficulty of anticipating what an agent will do next [39]. A separate body of research on human factors in security and automation shows that presenting a prompt does not guarantee that people understand or act on it, and describes broader forms through which people oversee automated systems [19, 23, 44]. Indeed, the mechanisms that agents deploy in practice all route decisions through the user,

Authors' Contact Information: Larissa Salerno, Chalmers University of Technology and University of Gothenburg, Gothenburg, Sweden, larissas@chalmers.se; Haoyu Gao, The University of Melbourne, Melbourne, Australia, haoyug1@student.unimelb.edu.au; Gregory Gay, Chalmers University of Technology and University of Gothenburg, Gothenburg, Sweden, ggay@chalmers.se; Alexander Serebrenik, Eindhoven University of Technology, Eindhoven, The Netherlands, a.serebrenik@tue.nl; Philipp Leitner, Chalmers University of Technology and University of Gothenburg, Gothenburg, Sweden, philipp.leitner@chalmers.se.

which has led to calls to treat agent security as a question of agent-human interaction rather than of enforcement alone [54]. However, little is known about how software practitioners themselves interpret and respond to individual permission requests from coding agents as a task unfolds in practice, including what they understand about a requested action, how they decide whether to grant it, and how they oversee an agent's behaviour once permission has been given.

We address this gap through a mixed methods study of how software practitioners understand agentic AI assistants and make and reflect on permission decisions from these assistants in their daily work. We conducted a two-phase sequential study consisting of an interview study followed by a survey. In the first phase, we interviewed software practitioners with experience using agentic AI assistants (e.g., Claude Code, Codex, Gemini CLI) about their use and understanding of these tools, their permission decisions, and their oversight practices. We qualitatively analysed the interview data to identify recurring behaviours, perceptions, decision factors, permission boundaries, and oversight practices. In the second phase, we developed a survey informed by the interview findings to validate the patterns identified in the interviews with a broader sample of software practitioners. Combined, our study addresses the following research questions:

RQ1. How do software practitioners understand the actions of agentic AI assistants?

We focus on the extent to which practitioners understand how these agents operate and act on their machines, with a particular focus on permission-related actions and requests. We find that practitioners build their understanding mainly from what they can observe and review, such as outputs, changes, logs, and reasoning traces, while still facing gaps in visibility and uncertainty about what agents do behind the scenes.

RQ2. How do software practitioners make decisions about permission requests from agentic AI assistants?

We look into how practitioners respond when they receive permission requests, the types of permissions they are willing to grant, and the factors behind these decisions. We find that permission decisions depend strongly on context, including the risk and scope of the action, whether it fits the task, and the environment in which the agent is running. Participants are generally more comfortable with bounded and low-risk access, while sensitive data, privileged access, destructive actions, and actions outside the intended scope form stronger boundaries.

RQ3. How do practitioners oversee and reflect on the actions and permissions of agentic AI assistants?

We investigate how practitioners monitor agent actions and permissions, respond to unexpected behaviour, and assess the transparency of these tools. We find that practitioners oversee agents in different ways, including setting constraints in advance, following execution as it happens, and reviewing the result afterwards, with the level of oversight changing with factors such as risk, task importance, trust, and time pressure.

These findings suggest that permission systems should do more than simply ask users to approve individual actions. Tools should make important actions and resource access easier to review, account for the fact that repeated approval may reflect fatigue rather than preference, and make the consequences and reversibility of actions clearer. They should also distinguish between what an agent is technically allowed to do and what the user actually intended it to do, while giving users ways to distinguish between denying a single action and ruling out an entire approach.

2 Related Work

2.1 Agentic AI in Software Development

Coding agents enable practitioners to automate implementation tasks. Systems such as SWE-agent navigate repositories, edit files, and execute tests in response to development tasks [60]. In a field study at Microsoft, Murphy-Hill et al. found that adopting Claude Code and GitHub Copilot CLI was associated with sustained increases in developers' merged pull requests [40]. These capabilities support different ways of organising development. In its original formulation, "*vibe coding*" involves delegating implementation through natural-language prompts without directly reviewing or editing the generated code [45]. In practice, they found that *vibe coding* involved iterative cycles of prompting, inspecting generated code, testing applications, and manually editing code. More recently, "*spec-driven development*" has emerged as a more structured workflow for guiding agent implementation, in which developers write or refine structured natural-language specifications that direct agents' work [2].

The use of AI in software development introduces challenges regarding productivity, understanding, and software quality. Becker et al. found that early-2025 AI tools slowed experienced developers working in familiar repositories despite perceived productivity gains [12]. Balepur et al. observed improved initial task completion but weaker code comprehension among students using coding agents [9]. Baltes et al. examined developers' discussions of AI "slop", identifying concerns about review burden, code quality, and the wider costs of low-quality AI-generated contributions [10]. Gao et al. found that AI-co-authored pull requests were merged faster and received fewer recorded comments and reviews than human-only contributions [24]. Choudhuri et al. further found that developers wanted AI support with clearly defined limits on its authority, while retaining human judgement and control over consequential decisions [16]. These findings motivate closer examination of how developers understand and oversee AI-assisted work.

2.2 Permission Mechanisms in Agentic AI

Permission mechanisms govern which actions agents may perform on a user's behalf. Michael and Roesner surveyed 21 permission system proposals and compared five commercial agents, finding that commercial systems often require choosing between burdensome manual approvals and automated permission decisions made by LLMs, with limited transparency and user control [39]. Kravchenko et al. proposed an information-flow control framework that checks whether tool calls comply with security policies before execution and verifies their outputs before admitting them into the agent's context. It can process untrusted data in disposable branches and return validated structured results while discarding the branch transcript, allowing the main agent to continue without incorporating the raw untrusted content [34]. Shukla et al. proposed Hedwig, which learns behavioural guidelines from developers' decisions and feedback to adjust when coding agents seek further input, accommodating changes in oversight preferences across tasks and over time [46]. Wu et al. conducted a user study to identify contextual factors shaping data-access permission decisions, then combined contextual information and prior decisions to predict future choices [57]. Their approach automates permission handling while accounting for differences in users' preferences.

Other work examines how the timing and granularity of these decisions affects their effectiveness. Zhou et al. modelled confirmation as a scheduling problem to place checkpoints inside a task, and found that 48 participants preferred intermediate confirmation over the confirm-at-end approach of existing systems [61]. Yan compared per-action approval, automated model review, and user-authored consequence policies across 113 participants supervising a simulated day of agent actions. Counter-intuitively, the user-authored policies blocked less overreach than either

per-action alternative, and most overreach actions were explicitly approved by a human at runtime [59]. Surveying 59 papers, 21 production systems, and 26 security plug-ins, Wang et al. argued that agent security is primarily an agent-human interaction problem, as the mechanisms industry deploys leave users caught between approval fatigue and uncontrolled agent autonomy [54].

Developers also guide agents through skills and repository context. Gao et al. found that skills are commonly copied, with behavioural constraints remaining largely stable as operational instructions are adapted [25]. Treude et al.'s preliminary investigation illustrates how developers encode privacy, fairness, and accountability requirements in repository context files [49]. Such artefacts express expectations before execution, while runtime permission requests ask developers to judge specific actions. These studies explain how developers express expectations for agents, but offer limited insight into how they interpret and respond to permission requests during execution.

2.3 Human Permission Decisions and Oversight of Automation

Research on usable security highlights the relevance of user attention, context, and interface design to how users respond to permission requests and security warnings [1]. Studies of Android permissions report limited attention and comprehension [23], alongside context-dependent preferences for resource access [56]. Research on browser warnings and security dialogues shows that presentation affects user responses [3, 14], while repeated exposure can reduce attention through habituation [51]. Research on automation further distinguishes appropriate reliance from misuse and disuse [35, 44]. Together, these studies suggest that providing a permission prompt does not ensure that users understand the request or assess its consequences.

Dhanorkar et al. interviewed developers and identified four forms of agent oversight: a priori control, co-planning, real-time monitoring, and post-hoc review [19]. Consistent with post-hoc review as a form of oversight, eye-tracking evidence shows that although developers do not review LLM-labelled code more thoroughly overall, they fixate longer on it and adapt their review strategy, revealing a gap between reviewers' stated intentions and their actual behaviour [30]. While the Android permission studies above examined access to protected resources, coding agent requests can involve commands selected by the agent as a task unfolds [39]. Developers must assess both the consequences of an operation and whether it fits the task they delegated. This creates a tension between delegating implementation and retaining enough understanding to authorise its individual steps. How software practitioners make these assessments when responding to individual permission requests remains insufficiently understood.

Altogether, this body of work shows what agentic coding tools can do and the challenges they introduce, early attempts to design permission mechanisms for agents, and general principles of human oversight of automation. However, no study has looked into how developers understand and decide on individual permission requests from coding agents as they arise during development. This paper addresses this foundational gap by studying how software practitioners understand and act on permission requests spanning file modifications, command execution, and data access and sharing, and how they oversee agent actions during delegated development.

3 Methodology

Figure 1 provides a visual overview of our research methods. We conducted this research as a sequential mixed methods study [47] with two steps: (1) a qualitative interview study, and (2) a confirmatory web-based survey intended to quantify the findings from the first step. This method follows established empirical research practice in software engineering, and has been used extensively in earlier work [8, 13, 17, 22, 53]. Eligible participants for both phases were software practitioners with experience using AI agents, and participation was voluntary and uncompensated

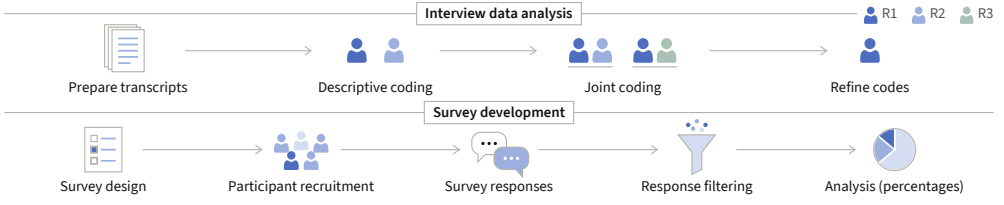


Fig. 1. Overview of the interview analysis and survey study, from transcript preparation and coding to survey design, participant recruitment, response filtering, and analysis. R1–R3 refer to the three researchers involved in the interview analysis.

throughout. Under the first author’s institutional procedures and legal requirements, neither phase of this study required formal ethical review.

3.1 Interviews

We recruited software practitioners through a combination of convenience and purposive sampling [11]. Our professional network provided an accessible pool, while we purposively sought variation in participants’ roles, experience, AI-agent use, and countries of work. Country variation was intended to capture potentially different regulatory contexts, although the study was not designed for country-level comparisons. Prospective participants completed a background questionnaire, and eligibility was assessed manually. We recruited 18 respondents meeting these criteria. Rather than a predefined sample size, we used saturation as a stopping criterion: few new codes emerged after P12, and P17 and P18 introduced no new codes, suggesting that code saturation had been reached by the final interviews.

Table 1 summarises the participants’ professional and AI-tool-use backgrounds.

We conducted semi-structured interviews between June and August 2026. Seventeen interviews were conducted online using Microsoft Teams, while one was conducted in-person and recorded using Teams.

The interview guide was developed after two trial interviews with one co-author and one industry contact, and included questions about what participants use AI agents for, how well they understood the actions taken by agents on their computers, and how they responded to permission requests. No substantive changes were made to the guide during data collection. The interview guide is available in our online appendix (see Section 8).

Interviews typically lasted approximately 30–45 minutes. One participant’s interview was substantially longer and was continued in a second session on a separate day. Interviews were conducted in English, Portuguese, or Chinese by an author fluent in this language. Non-English interview transcripts were machine-translated into English and subsequently checked by an author fluent in the source language as well as English. Participants provided written informed consent before each interview. We did not report participants’ names or company identities and do not make the interview transcripts publicly available to protect participant confidentiality, following recommendations for ethical interview research in software engineering [48]. The consent form is available in our online appendix (see Section 8).

We analysed the interview data using a multi-stage coding and categorisation process, drawing on open coding and constant comparison procedures described in Socio-Technical Grounded Theory [28]. We identified excerpts relevant to the interview topics and summarised them using descriptive codes, which were refined into shorter focused codes covering participants’ understanding of agent behaviour, responses to permission requests, and oversight practices. To align their coding

Table 1. Participant background characteristics for the interview ($N = 18$) and survey ($N = 115$) samples. Filled dots indicate interview participant responses, n gives interview counts, and SP% indicates survey percentages. A dash indicates that the item was not asked in the survey. Participants could select more than one AI assistant tool. All other questions were limited to a single selection.

Question/Options	P01	P02	P03	P04	P05	P06	P07	P08	P09	P10	P11	P12	P13	P14	P15	P16	P17	P18	n	SP%
<i>Which country do you work in?</i>																				
Sweden	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	9	27.8%
Brazil	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	25.2%
China	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	0.0%
Netherlands	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	2	3.5%
Australia	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	1	2.6%
Other	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	0	40.9%
<i>How many years of professional experience do you have in IT?</i>																				
0–2 years	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	2	9.6%
3–5 years	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	20.0%
6–10 years	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	11	27.0%
11–15 years	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	2	22.6%
16+ years	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	0	20.9%
<i>What is your seniority level?</i>																				
Junior	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	–
Mid-level	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	6	–
Senior	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	9	–
<i>How large is the company you work for?</i>																				
Startup (1–50)	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	1	–
SME (51–500)	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	4	–
Large enterprise (500+)	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	13	–
<i>How long have you been using AI assistant tools?</i>																				
Less than 6 months	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	5.2%
6–12 months	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	7	33.9%
1–2 years	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	7	41.7%
More than 2 years	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	1	19.1%
<i>How often do you use AI assistant tools?</i>																				
Daily	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	14	–
Weekly	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	–
Occasionally	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	1	–
<i>Which AI assistant tools do you use? (tick all that apply)</i>																				
Claude Code	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	12	–
GitHub Copilot	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	7	–
Codex	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	4	–
Gemini CLI	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	–
Cursor	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	2	–
Other tools	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	3	–

approach, two authors first coded two transcripts jointly. One author then coded ten transcripts independently and the other five, while the first author and a third, senior author jointly coded one. To check the consistency of the coding, the third author also independently re-coded two transcripts, and the authors compared and discussed differences. The first author then reviewed the focused codes across all participants, iteratively merging, renaming, or consolidating codes that expressed similar ideas to reduce overlap and produce a final set of codes. Related final codes were grouped into higher-level categories and dimensions, developed from the data rather than defined in advance, which we used to structure the results. Our full codebook is available online (Section 8).

3.2 Survey

Following the interview analysis, we developed a confirmatory survey to explore whether the patterns identified in the interviews were also recognised by a broader sample of software practitioners. The survey covered the same topics as the interviews, i.e., how participants understand AI agents, how they respond to permission requests, the factors that influence their permission decisions, and their perceptions of permissions and agent transparency. Response options were based on the final interview codes. For categorical questions, we kept all categories, and for permission boundaries, the specific permission or action types. For other topics, we selected recurring, clearly

distinct findings that could be turned into structured survey items, while leaving out highly specific, overlapping, or difficult-to-operationalise findings to keep the survey manageable. Items included multiple-choice questions (both select one and select all that apply), statement selections, and several with an optional open-text “Other” response. One instructed-response attention check was included to help identify inattentive responses [38]. The survey was designed to take approximately 8–10 minutes to complete. We pre-tested the survey with three PhD students, whose feedback led us to revise wording, add the attention-check item, and remove one question on unexpected agent behaviour because the predefined response options felt too restrictive for that topic.

Participants were first presented with a page describing the purpose of the study, the expected survey length, the anonymous nature of participation, how their responses would be used, and how to contact the research team. They were informed that they could stop the survey at any time before submitting their responses, and that continuing with the survey indicated their consent to participate. We administered the survey through Typeform [50]. Similar to interview transcripts, we do not make individual-level survey responses publicly available and instead report findings in aggregate form. Our survey instrument is also contained in our online appendix (Section 8).

We recruited participants using a combination of convenience, purposive, and snowball sampling, distributing the survey through the research team’s industry contacts, relevant Reddit communities focused on software development and AI agents, LinkedIn, Bluesky, and Mastodon, and by asking interview participants to share the survey within their organisations or professional networks. We considered the online channels appropriate for reaching software practitioners, as previous work has identified active software development communities on Reddit and shown that software engineering practitioners engage with research on LinkedIn [5, 58]. Eligibility was assessed from participants’ self-reported job title or role and their reported length of AI-agent use. Responses from participants who did not meet these eligibility criteria, failed the attention check, or were incomplete were excluded from the analysis. We also included a screening question asking whether participants had noticed AI agents requesting permission before taking actions or accessing resources. Participants who answered “No” were redirected to the end of the survey, while those who answered “Yes” or “I am not sure” continued with the permission-focused questions.

We, therefore, use two survey samples in the analysis: 115 participants for questions appearing before and including the screening question and 107 participants for the permission-focused questions. Column *SP%* in Table 1 reports background characteristics of survey respondents. Because the survey collected fewer background items than the interviews, percentages are shown only for country, professional IT experience, and length of AI-agent use. For country, “Other” groups all countries not listed individually in the table, and all survey respondents reported a country. We calculated response frequencies for each survey question. Percentages are based on the number of participants eligible to answer each question. Responses entered through the optional open-text “Other” fields were reviewed manually. As relatively few were provided (21 responses across 7 questions), relevant responses were summarised in the text. The figures reporting survey results were generated with Python scripts written with the assistance of an AI tool (Claude); all plotted values are computed from the survey response counts and were verified by the authors against the survey analysis. The survey results were organised around the same three research questions as the interview findings, and we compared the survey responses with the interview findings to see whether similar behaviours and perceptions were reported by the broader sample.

4 Results

We organise the results around the three research questions. We find that practitioners’ understanding of agent behaviour is uneven and largely confined to what they can directly observe, such as outputs, changes, and reasoning traces (RQ1). Because this understanding is partial, permission

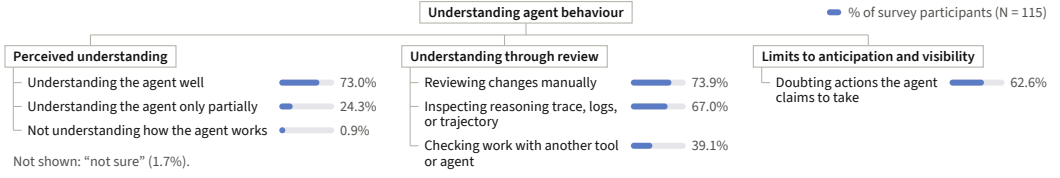


Fig. 2. Participants’ perceived understanding of agent behaviour, how they review agent work, and the limits to anticipating agent actions (N = 115).

decisions rely on heuristics such as scope, risk, familiarity, and task fit rather than a full assessment of what an agent is about to do (RQ2). These heuristics, together with the visibility gaps that limit understanding, in turn shape how practitioners oversee agents once permission has been granted, producing oversight strategies that shift with task importance and risk, task complexity, demonstrated reliability, and time pressure (RQ3). The labels in the figures in the Results section represent findings identified in the interviews, while the percentages show how many survey respondents selected the corresponding survey item. Only interview findings that were included in the survey are shown in these figures.

4.1 Understanding of Agentic AI Assistant Behaviour (RQ1)

We examined participants’ understanding of agent behaviour, which helps explain their approaches to permission and oversight. We organise our findings into four categories, and Figure 2 summarises the three categories directly covered by the survey. *Perceived understanding* represents participants’ self-assessment. *Understanding through review* describes how participants improve or verify that understanding. *Trust and sharing judgements* reflects how participants reason about information shared with agents and whether they trust agents or their operational environment. Finally, *limits to anticipation and visibility* outlines situations where participants struggle to predict, observe, or maintain visibility of agent actions.

Perceived understanding. We distinguish three levels of understanding: full, partial, and none. Almost half our interviewees described partial or no understanding. Conversely, survey respondents judged their understanding substantially more positive: 73.0% believe they have full understanding, 24.3% partial, and only 0.9% report no understanding (1.7% were unsure).¹

Confidence is often confined to what participants can directly observe, and what happens behind the scenes is less clear. Some interviewees feel they understand systems better when they can see how the underlying process works, but are less sure how tools like Claude Code decide what to do, when to request permission, or how data is used:

“Under the hood, no, not not really. I only understand the output of what it’s giving me and only observable things” - P05

Technical knowledge does not necessarily make these behaviours clearer:

“I understand some of the basic principles of machine learning [...] but for example, what Visual Studio Code is doing under the hood [...] is something that I need to understand better” - P16

These findings confirm concerns raised in prior work: greater agent autonomy can make it harder for users to understand both the resulting code [9] and the agent’s behaviour [15].

Understanding through review. Participants often try to understand what the agent was doing by examining its work (output) during or after a task. P03, for example, gains confidence by inspecting

¹ However, recall that the survey results reflect *self-assessed* understanding, which is typically higher than real understanding of a topic [18].

the agent’s trajectory, and checking which tools and scripts it invokes. P14 similarly relies on seeing the changes the agent makes and becomes more cautious when they are not shown:

“If I find that it claims to have done something but doesn’t show me the changes, that’s when I step in” - P14

P17 similarly finds it easier to understand when they can follow their agent as it works:

“If [...] I only see the final result. I find it hard to understand what it did” - P17

Others apply tools or other agents to double-check work. P14 asked a second agent to assess another agent’s conclusions, while P15 described using another agent or tooling to check generated code for code smells.

73.9% of survey respondents manually review the agent’s changes, 67.0% check the reasoning trace, logs, or trajectory, and 39.1% use another tool or agent. Textual responses mentioned skimming changes to catch cases where the agent had over-reached and running code in test environments.

Participants figure out what the agent has done by checking its work afterwards, rather than already knowing how it works up front, similar to the real-time monitoring and post-hoc review described by Dhanorkar et al. [19]. Here, these practices served both as review and to build understanding.

Trust and sharing judgements. Interviewees considered where their data might go and their comfort in sharing it. For some, this judgement comes before any question of permissions:

“Before asking something from Claude or anything else, I consider whether I’m okay sharing this data [...] If it’s something sensitive, of course, I just don’t use it for that” - P05

Survey respondents share similar feelings: 61.7% would never grant an agent access to sensitive data, and 58.9% said the same for email or other personal information (see Figure 4). This is consistent with work showing that data sensitivity strongly shapes access decisions for agents [57].

Once P05 considers a project safe to share, they are less concerned about what happens “under the hood”. For other interviewees, how data is used still matters. P08 considers where their code is sent and whether it could be used for model training, while P11 feels more comfortable when models are hosted within the company’s own infrastructure. Sharing decisions depend not only on data sensitivity, but also where it is processed and how it will be handled. Dhanorkar et al. similarly found that developers were concerned about data sent to third-party platforms [19].

Limits to anticipation and visibility. Reviewing agent actions does not mean participants can anticipate or verify actions. 62.6% of survey respondents expressed doubt in actions agents claimed to be taking. One response said that, despite understanding the technical setup, they sometimes doubt whether agents do what they claim and would step in to guide them. P12 also described approving an action based on one expectation, only for the agent to do something different.

Visibility varies between tools. P04 feels they have more control over permissions and data flow with open-source frameworks than with Claude Code, while P11 feels there is little visibility into background activity, especially when agents work simultaneously:

“What the sub-agents are actually doing and what instructions they are using, honestly, you can’t really perceive that” - P11

This challenge may become more pronounced as coding tools increasingly support subagents and background tasks that run in parallel [6].

Takeaway: Software practitioners build understanding through what they can observe and review: changes, reasoning traces, logs, and outputs. Although a majority feel they understand agent output, they express doubts regarding visibility of individual decisions, trust, and access to sensitive or personal data.

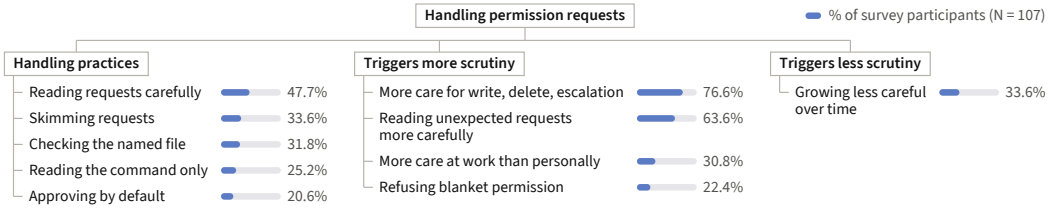


Fig. 3. How participants handle permission requests, including their handling behaviours and the conditions that lead to more or less scrutiny.

4.2 Permission Decision-Making (RQ2)

Because participants' understanding of agent behaviour is often partial and limited to what they can observe (RQ1), permission requests become a key point at which they can actively shape what an agent is allowed to do. We now discuss how software practitioners react to agent permission requests.

4.2.1 Handling Permission Requests. We organise findings regarding review and approval practices (Figure 3) into three groups: *handling practices* reflects participants' reactions to permission requests, while *triggers more scrutiny* and *triggers less scrutiny* describe factors that make participants more or less careful.

Handling practices. Permission requests are handled in various ways. 47.7% of our survey respondents believe they read permission requests carefully. P12 describes how tiring this can become:

“[I] pay attention to what it's asking permission for, and then manually approve or not. This really is exhausting.” - P12

Reviewing a request does not necessarily mean reading it in full. P03, P05, and P14 skim requests, as do 33.6% of survey respondents. Participants also often focus on particular aspects of a request: 31.8% check the named file, while 25.2% focus on the command or title. P05 reads more carefully when something is unfamiliar.

Some interviewees approved requests by default, an approach also reported by 20.6% of survey respondents, while others prefer blanket permissions. However, some reported using blanket approvals while still keeping certain tools, commands, credentials, or sensitive code off-limits. These approaches reflect a trade-off between control and efficiency, as reviewing each request takes time and attention. As stated by P17:

“Otherwise it kind of defeats the point of being faster, because every 10 seconds I have to click something” - P17

Triggers more scrutiny. Participants increase their scrutiny when requests carry more risk, are unexpected, or happen in less familiar or controlled settings. P09 highlighted write requests:

“When it's trying to write, I pay much more, much closer attention, especially if it's something that I don't expect” - P09

P05 and P07 also noted that they apply more scrutiny to unexpected requests. In the survey, 76.6% said they check write, delete, or privilege-escalation requests more carefully, while 63.6% do so when a request is unexpected. Context matters, too: P02, P04, and P16 are more careful at work than on personal projects, while P12 is more cautious outside a controlled environment. 30.8% of survey respondents also said they are more careful at work than on personal projects. In addition, 22.4% of survey respondents refuse blanket permission and prefer to be asked each time. This preference for case-by-case handling is also visible when users are offered an alternative: given the chance to set standing rules in advance, participants in an earlier study chose “ask” for most of them, returning the decisions to runtime [59].

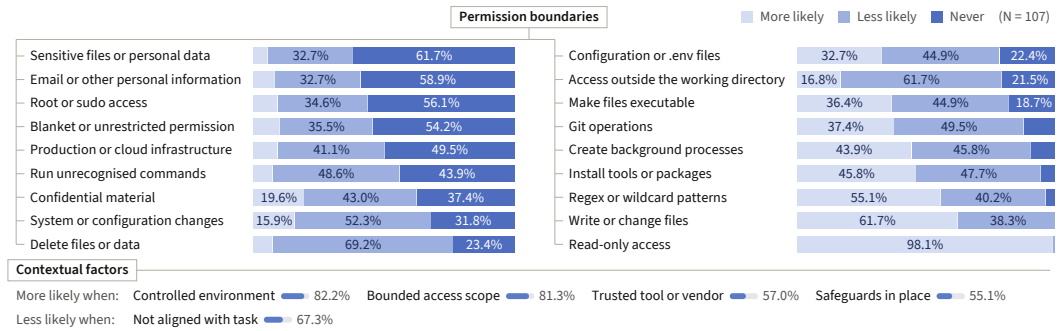


Fig. 4. How participants assess different permission request types, showing which requests they are more likely, less likely, or would never grant compared with other permission requests, and the contextual factors that make them more or less likely to grant permission.

Triggers less scrutiny. The main factor that triggers participants to be less careful is familiarity: in the survey, 33.6% of respondents said they have become less careful about permission requests over time (and, given that this is self-reported, this number is likely an underestimation). P04 describes clicking through repeated prompts without looking at them closely, while P17 sometimes moves straight to bypassing permissions.

Having to deal with many requests can have a similar effect: at some point, there are simply too many to review individually.

“There’s no way that I can actually go through all the permission requests” - P03

This is similar to habituation to repeated security warnings [51] and approval fatigue reported for Claude Code, where users approve 93% of permission prompts [29]. Wang et al. describe a trade-off between cognitive burden and security guarantees that current mechanisms leave unresolved [54].

P03, P06, and P13 also pay less attention when working in a controlled environment. They rely on the sandbox around the agent, giving agents more autonomy within the defined workspace while requiring additional approval beyond it [20, 42].

4.2.2 Permission Boundaries. We organise findings on the types of permissions participants are willing to grant (Figure 4) into three groups: permissions they are *more likely to grant*, *less likely to grant*, or *would never grant*, compared with other permission requests. Participants do not judge permissions by permission type alone: the same permission can be acceptable or unacceptable depending on its scope, task, and environment, consistent with recent work showing that permission preferences for AI agents change with context [57].

More likely to grant. Participants are most willing to grant permissions that feel limited and low risk. Importantly, many interviewees are more comfortable with read-only permissions, as were a large majority (98.1%) of survey respondents. P03, P11, and P18 prefer agents to stay within a particular file, folder, or working directory.

In the survey, 82.2% also said they were more likely to grant permission when working in a controlled or isolated environment, and 81.3% when the access scope was clearly bounded.

Interviewees are more willing to grant permission when the requested action matches the context of what they ask the agent to do:

“If it makes sense for the task it’s supposed to do, then I would be likely to give it [...] But if it’s trying to ask for permissions for do something that seems very unrelated, then I would probably decline it” - P07

Trust also matters: P02 and P09 say they become more willing to grant permissions as they gain experience with particular agents or models, while P13 points to vendor reputation and safeguards around dangerous operations. In the survey, 57.0% said they were more likely to grant permission when they trusted the tool or vendor based on experience or reputation, while 55.1% were more likely to do so when safeguards constrained risky actions.

Less likely to grant. Caution increases when permissions can lead to more consequential changes or when the request does not seem to fit the task. P01, P09, and P13 are less comfortable with write-related permissions. In P09's case, they are much more careful when the agent can change something on the machine. For actions such as installing drivers or changing system settings, they would be *"very, very extremely careful"* and would probably make the change themselves.

Deletion makes this distinction especially clear. P12 is cautious about deletion requests and describes an extreme case where an agent, despite operating in a confined environment, asked to delete the C drive.

P12 asks *"why do you want to delete this?"* and, if deletion is justified, notes a preference to do it themselves. They also rely on backups and Git to make mistakes recoverable. Similarly, 69.2% of survey respondents said they are less likely to grant permission to delete files.

P07 and P14 expressed caution about requests that seem unrelated to what they asked the agent to do. 67.3% of survey respondents are less likely to grant a request when it does not align with the task. This suggests that participants consider not only the type of permission, but also whether the action makes sense in context and how easily its consequences can be reversed. Other concerns raised include access outside the working directory, sensitive files, Git operations, package installation, unfamiliar commands, and regex or wildcard operations.

Would never grant. For some permissions, participants establish clear boundaries. Sensitive or personal information is one of the strongest. 61.7% of survey respondents said they would never grant access to sensitive data, while 58.9% said the same for email or other personal information. Interviewees similarly rule out credentials, passwords, tokens, confidential company data, and personal files. P05 is particularly direct:

■ *"No, never personal information [...] never anything personal"* - P05

System-level actions are another clear boundary. In the survey, 56.1% said they would never grant root or sudo access, and 23.4% said the same for deletion. P16, for example, rejects deletion requests, elevated commands and describes critical system locations as a *"no-no"*:

■ *"Anything with [sudo] [...] You cannot sudo anything"* - P16

For others, the limit is how far the agent can reach. Participants mention Git operations, access outside the working directory, and production or cloud environments.

■ *"Any kind of settings, no [...] I wouldn't like [Claude] to update anything in my machine or anything related with the project or even my local environment"* - P18

In some cases, these boundaries are stricter than those enforced by current tools. GitHub's coding agent, for example, can create commits and push to a dedicated branch while preventing direct pushes to the default branch [27]. Some participants, however, treat Git operations themselves as beyond what they are willing to let an agent do autonomously.

Takeaway: Practitioners make permission decisions with scrutiny ranging from careful review to default or blanket approval. How closely they review requests depends on the type of action, whether it is expected, the environment, and repeated exposure to similar requests. They are more comfortable with bounded, low-risk, and task-relevant access, while sensitive data, privileged access, destructive actions, and access beyond the intended scope form stronger boundaries. These heuristics compensate for limited visibility and concerns over data handling identified in RQ1.



Fig. 5. How participants oversee agent actions, including the strategies they use, factors that shape the level of oversight, and the role of the agent’s reasoning trace.

4.3 Reflection on Agent Actions and Oversight Strategies (RQ3)

4.3.1 Oversight of Agent Actions. We organise our findings regarding how participants follow agents and how they try to stay in control of what the agent does (Figure 5). *Oversight approaches* show how interviewees monitored or constrained agents. *Factors shaping oversight* capture when they paid more or less attention. *Reasoning as an oversight mechanism* looks at how interviewees used, or chose not to use, the reasoning and execution information shown by the agent.

Oversight approaches. Software practitioners differ in when and how they oversee agents. Some stay involved while the agent works. P05, P06, and P07 describe interrupting, redirecting, or stopping the agent when necessary, an approach reported by 69.2% of survey respondents. In those cases, participants keep a close eye on the agent as it works.

Other interviewees check the result once the agent has finished, as do 52.3% of survey respondents.

“Usually, I let it run and check the output [...] at the end. What matters to me is [...] the outcome” - P11

Some try to constrain what the agent can do before it starts, for example, establishing guardrails through configuration or skills, an approach also reported by 50.5% of survey respondents.

“We also put limits on the AI, in advance [...] there are directories you can’t touch, or commands you can’t play with, so it is constrained up front” - P15

Some go further and let the agent run with little active monitoring, similarly reported by 40.2% of survey respondents. P10 is comfortable doing this for unit testing within a working directory, while P17 takes more of a “hope for the best” approach:

“Just cross[ing] my fingers, just hope it does the right thing.” - P17

P02, P05, and P07 review changes step-by-step, as do 30.8% of survey respondents. A further 26.2% reported watching which files the agent reads. Where practitioners position themselves along this range determines when they are able to catch a mistake. Prior work on checkpoint placement found that users preferred confirming intermediate steps over confirming only at the end, because a single early error can otherwise cascade and force a restart [61].

Factors shaping oversight. How closely participants oversee agents depends on the task and situation. Several interviewees mentioned paying more attention when the work is important or risky, consistent with 70.1% of survey respondents. Some interviewees also mentioned reviewing complex tasks more carefully.

As confidence in the agent increases, participants tend to step back. P03 and P07 pay less attention after the agent has performed reliably several times, which is also reported by 31.8% of survey respondents. Under delivery pressure, P08 describes almost abandoning review:

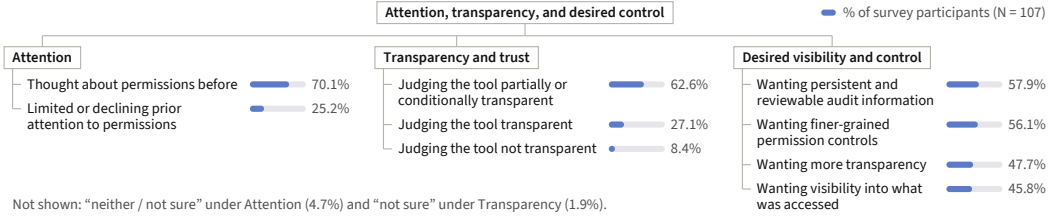


Fig. 6. Participants' attention to permission requests, perceptions of agent transparency, and desired forms of visibility and control.

"I need to deliver this in 3 hours [...] I'm not going to read anything. Like, 'just generate it for me.' And then later I test it, it worked, ship it" - P08

Levels of oversight, therefore, change with risk, task complexity, trust, and time pressure. Many of these are the same factors that shaped scrutiny of individual permission requests (RQ2): participants apply a risk- and context-sensitive heuristic when deciding whether to grant a request *and* deciding how closely to watch what happens afterwards. This aligns with prior work showing that comfort with agent autonomy can depend on the task and shift over time [46].

Reasoning as an oversight mechanism. Interviewees also track what agents are doing by looking at the reasoning they show, but several said they do not always read it closely, skipping or giving up on it because it takes too much time or attention, or simply produces too much information. Similarly, 42.1% of survey respondents said they sometimes reduce or skip reasoning review because of time, fatigue, or information overload.

P11 noted that following it too closely can defeat the purpose of using an agent, shifting them from *"a user benefiting from AI"* towards *"more of a maintainer of AI"*. Even when reasoning is shown, it is not always easy to use. P07 mentioned one instance where reasoning disappeared once the final answer was produced, while P17 finds the operation history difficult to validate. Although the reasoning shown by AI agents is not necessarily the model's full chain-of-thought, prior work suggests that it can help users monitor what the agent is doing [41]. Our findings show that this only works when users can actually keep up with, and make sense of, what they are shown.

4.3.2 Attention, Transparency, and Desired Control. Interviewees offered diverse opinions regarding *attention* to permissions, perceptions of *transparency and trust*, and preferences for greater *visibility or control* (Figure 6).

Attention. Some interviewees described paying close attention to permissions, which was also noted by 70.1% of our survey participants.

Others felt their attention was limited or had declined over time, as did 25.2% of survey respondents. P12 recalled that reading "horror stories" about agents has made them more cautious. P03 described the opposite: as permission requests became familiar, they started paying less attention.

"I was concerned about the permission quite a bit [...] But after a while, I feel like, okay, it's asking for similar permissions. So after that, I stopped caring" - P03

As previously observed, repeated requests can become tiresome and time pressure reduces attention further. P18 said they often approve requests without much thought because they are *"in a rush to deliver something or to finish something"*.

When asked what they would pay more attention to, participants mentioned dependency installation (P03), accessed resources (P08), audit logs (P09), and intermediate steps (P18). A risky

request can make participants stop and think, but familiarity or time pressure makes them more likely to approve and move on.

Transparency and trust. Four interviewees consider AI agents transparent, three see them as partly or conditionally transparent, and six do not consider them transparent. Most respondents in our survey also describe agents as partially or conditionally transparent (62.6%), while 27.1% consider them transparent and 8.4% do not.

Even when parts of the interaction are clear, participants still see gaps. P03, for example, finds individual permission messages easy to understand, but once permissions are bypassed, they are no longer sure what the agent is doing. P11 has a similar problem, even though they find other information about the agent’s activity easier to follow.

Trust does not always depend on seeing everything the agent is doing. P08 puts this directly:

■ *“Yeah, I trust the tool, but that doesn’t mean it does what it claims to, right?” - P08*

They wonder which files or resources the agent might access as this is difficult to follow from the reasoning. Other interviewees trust the agent more because of experience or safeguards. P06 said they have become more trusting as the tools have matured, while P03, P11, and P18 point to controlled environments or employer-provided safeguards. Participants can, therefore, trust an agent despite gaps in visibility, particularly when other safeguards give them this confidence.

Desired visibility and control. When asked what they would change, interviewees desired a clearer view of what the agent has done and more control over what it is allowed to do. Several wanted a record of the agent’s activity that they could review afterwards.

■ *“Everything to always be logged somewhere [...] the commands that were run, so there could be an audit trail.” - P10*

Similarly, 57.9% of survey respondents want persistent information or audit logs.

Some interviewees want more control. P05, P08, and P11 want more detailed ways to limit what agents can access or do, while 56.1% of survey respondents said they want finer-grained controls over permissions or resources. Currently, AI agents are required to piggy-back on operating systems permission models, which have not been designed for fine-grained access control. One open-text respondent explicitly linked this kind of control to reduced monitoring, saying they wanted agents constrained to selected folders and tools so they would not need to pay attention constantly. P05 also suggests clearer risk levels and project-specific permission settings, while P11 wants a persistent place to see which permissions are active and how files and information are being used.

More broadly, 47.7% of survey respondents want greater transparency about the agent’s activity. Other interviewees want to see more clearly what the agent has accessed, something 45.8% of survey respondents also ask for. P08, for example, imagines a visual view of the file system showing *“where it went, where it passed through, what it saw”*.

These requests are more specific than simply asking for “more transparency”: participants want a record of what the agent has done and clearer ways to set limits on what it can do. Similar features are beginning to appear in current systems. OpenAI, for example, describes logs of Codex activity alongside managed sandbox policies that limit where the agent can act [42].

4.3.3 Examples of Unintended Agent Behaviour. The scrutiny and boundaries described above shape what an agent is permitted to do, but not necessarily whether it behaves as expected. We therefore asked interviewees whether they had experienced any permission-related unexpected behaviour when using AI agents.

P12 described an experience where this was particularly noticeable. While debugging with Codex in a mode where the agent was supposed to ask for permission, they refused several requests. After a while, the agent stopped asking but still continued its work. P12 felt that the agent had found a way around what they had been refused.

“It ignored the instructions I’d given it [...] It worked around what I hadn’t allowed it to do, I found that pretty bizarre” - P12

Continuing after a denied request does not necessarily mean that the agent bypassed the permission. Agents may instead find an alternative way to complete the task [29]. In P12’s case, the concern was that they could not tell whether the alternative still respected the boundaries they had set, a behaviour also noted in Anthropic’s cybersecurity evaluations [7].

Interviewees also noticed unexpected behaviour after granting permissions. P06 recalled giving an agent access to an entire workspace while asking it to change one project, but the agent also changed other projects. P18 similarly found that giving full permission sometimes led the agent to do more than requested, including creating extra files and changing the README. In these cases, the agent did not necessarily go beyond the permissions it had been given, but it did more with those permissions than interviewees expected, similar to the unintended edits and scope drift that Hedwig is designed to address [46].

Takeaway: Software practitioners oversee agents in different ways, including setting constraints in advance, following execution as it happens, and reviewing the final result. How closely they do this depends on task importance, perceived risk, demonstrated reliability, and time pressure. Unexpected behaviour and gaps in visibility can make oversight difficult, leading participants to want clearer audit information and finer-grained control mechanisms.

5 Implications

More transparency does not always make oversight easier. Software practitioners often rely on changes, logs, trajectories, and reasoning traces to understand what an agent has done. Both our interviewees and survey participants described giving up on reviewing this information because of time pressure, fatigue or information overload. Related work has also reported disruption from noisy bot activity [55] and sources of confusion during code review [21]. At the same time, they still want persistent audit information and clearer visibility into what resources the agent has accessed. Therefore, transparency should not just be showing more information, but should be about making the important parts easier to find. Tool vendors could make this information easier to use by showing users what actually matters, such as important actions, unexpected behaviour, and which resources the agent accessed.

Repeated approvals may reflect fatigue and delivery pressure, not preference. Previous approvals do not always reflect a deliberate choice. Participants described becoming less careful over time, clicking through familiar prompts, and sometimes skipping review when under time pressure. Repeated approval may therefore reflect habit, fatigue, or a need to keep moving rather than growing trust. This matters for adaptive permission approaches that use past decisions to predict or adjust future permissions. Current tools already recognise part of this problem: Claude Code’s Auto Mode was introduced partly to reduce approval fatigue [29], while Copilot CLI and Codex distinguish one-off approvals from choices that grant broader permission for a session or longer [26, 42]. As permission handling becomes more automated or personalised, repeatedly clicking “allow” should not by itself be treated as evidence that the user wants the system to make the same decision on their behalf in the future. This reading is consistent with the argument that current approval mechanisms force a trade-off between cognitive burden and security, leaving users between approval fatigue and uncontrolled autonomy rather than expressing preference [54].

Potential consequences and reversibility shape permission decisions. Our participants see write, deletion, and system-level permissions as risky, but the type of action is not the only thing that matters. They also consider what can go wrong and whether the action can be undone. For example, deleting something that can easily be restored is very different from making an irreversible system change. Tool vendors could make recoverability visible in permission prompts by showing

whether an action can be rolled back and what recovery mechanism is available. Requests with irreversible or difficult-to-recover consequences could then receive stronger warnings or require additional confirmation. This would make reversibility part of the permission decision, rather than treating all actions of the same type as equally risky.

Agents can respect permission boundaries while exceeding the scope of the task. Software practitioners do not usually judge actions and permissions in a “general” sense, they care about whether they fit the task they have in mind. Several participants described agents doing more than expected while still staying within the workspace they had allowed them to use. The problem is not simply that agents cross permission boundaries. They can stay within those boundaries and still go beyond what the user *intended*. Current permission systems can already restrict where an agent may act and, increasingly, use the user’s instructions when deciding whether an action should proceed or not. Yet our participants still experienced agents doing more than they had asked while remaining within those boundaries.

What does “deny” mean when an agent can find another way? By design, agents attempt to find alternative paths after a permission request is denied. One interviewee described this directly: after refusing several permission requests, the agent continued using another approach, and they could not tell whether it still respected the boundary they had tried to set. While this is often desirable, e.g., when an agent can solve a task without accessing sensitive information, this behaviour can feel like overreach if the problem was less the specific permission and more the general intent of the action. In these cases, continuing to push for an action that the user felt they already said “no” to can be a severe violation of trust. Tool vendors need to provide better instruments for users to distinguish between denying specific actions and disallowing an entire solution path, e.g., to distinguish between “you are not allowed to read my passwords file” versus “do not attempt any action that requires you to have access to my password”.

6 Validity of the Findings

We discuss limitations separately for the interview and survey studies, following recommendations to align validity criteria with the study design [52]. For the interviews, we use Lincoln and Guba’s criteria of credibility, transferability, dependability, and confirmability [37]. For the survey, we discuss threats related to sampling, measurement, and response quality, drawing on established guidance for survey research [32, 33, 38].

6.1 Interview Study

Credibility. Our findings are based on self-reported experiences rather than direct observation or interaction logs, so participants may not always remember or report events accurately. Where a participant’s statement was ambiguous, we followed up with them to clarify our interpretation. In addition, researchers fluent in the source language checked all non-English transcripts against the original recordings, including every quotation used in the paper.

Transferability. Both phases used voluntary and non-probabilistic sampling (i.e., professional contacts, online communities, social media, and snowball sampling). Therefore, the sample may include more AI-agent users with strong experiences or opinions about permissions. The findings should hence be interpreted in relation to the characteristics of this sample and study context rather than assumed or transfer directly to all software practitioners who use AI agents. Finally, the study was conducted in 2026, and agentic AI tools change quickly, so some behaviours and concerns may change over time.

Dependability. Qualitative analysis involves interpretive judgement, and differences in interpretation are expected. We therefore involved multiple researchers to make coding decisions explicit and

discuss them throughout the analysis. The first and second authors coded two transcripts jointly. The first author then coded the English- and Portuguese-language transcripts, while the second author coded other English- and Chinese-language transcripts. The last author independently re-coded two transcripts, after which differences were discussed and resolved through consensus. The final codes were iteratively merged, renamed, and organised into categories, which were reviewed and agreed on by the co-authors. We maintained an audit trail by retaining earlier codebook versions, coding files, and related analysis material (included in the online appendix) to make the development of the analysis traceable.

Confirmability. The first author led the final consolidation of the interview codes, reviewed survey open-text responses, and selected the quotations used in the paper. These choices may reflect their interpretation of the data, so the other authors reviewed the final categories and checked the quotations used to support the findings. We kept links between codes and interview data throughout the analysis and provide the final codebook, survey questions, and survey analysis in the appendix, offering partial traceability from the reported findings to the codes. We did not keep a formal reflexive journal, although interpretive decisions were made explicit through joint coding, independent re-coding, and co-author review. We also conducted an additional audit of a sample of reported findings with a researcher who was not involved in the analysis. The auditor traced the findings through the final codes, coded excerpts, and transcript context. Some excerpts were initially difficult to interpret without the corresponding interview questions, so these were also provided as contextual information. After this was provided, the auditor did not identify substantive issues with the reported interpretations. The full transcripts cannot be shared for confidentiality reasons, which limits other researchers' ability to inspect the raw data and assess the link between the data and our interpretations.

6.2 Survey Study

Sampling and generalisability. The survey was designed to collect the information needed to address our study objectives without adding unnecessary questions [31]. We deliberately collected fewer background variables in the survey to keep it focused and limit participant burden. As a result, we cannot compare responses by tool, seniority, organisation size, or other factors.

Measurement and comparability. The survey provides another view of the interview findings, but the two phases are not directly comparable. Survey respondents chose from predefined options, while interview findings came from semi-structured responses. Also, some interview findings were left out to keep it manageable. Two authors reviewed the mapping between interview findings and survey items, and we included "Other", "None", or "Not sure" where appropriate. We therefore use the survey to see whether the same patterns appeared in a broader sample, not to directly compare their prevalence [47].

Response quality and analysis. For the survey, exclusion criteria were established before substantive analysis, and responses were manually reviewed for duplicates and suspicious response patterns in addition to the attention check.

7 Conclusion

We studied how software practitioners understand AI agents, make permission decisions, and decide how closely to monitor them through interviews and a follow-up survey. We found that people often understand agents mainly through what they can see, and that permission decisions depend on scope, risk, the task, and the environment. How closely they monitor the agent also changes depending on how risky the task feels, how much they trust the agent, and practical factors such as time and fatigue.

Our findings show that permissions are not just about whether an action is allowed. Users also care about whether the action matches what they wanted, whether it can be undone, and whether they can work out what happened afterwards. Permission systems should separate what an agent may do from what the user actually asked for, keep boundaries in place when the agent changes its plan, and make important actions easier to review without constantly interrupting the user.

8 Data Availability

An anonymised replication package containing the study materials, interview coding artefacts, survey-to-code mapping, and aggregate survey results is available at: <https://anonymous.4open.science/r/ai-agents-permission-replication-1FA6/>.

References

- [1] Alessandro Acquisti, Idris Adjerid, Rebecca Balebako, Laura Brandimarte, Lorrie Faith Cranor, Saranga Komanduri, Pedro Giovanni Leon, Norman Sadeh, Florian Schaub, Manya Sleeper, et al. 2017. Nudges for privacy and security: Understanding and assisting users' choices online. *ACM Computing Surveys (CSUR)* 50, 3 (2017), 1–41.
- [2] Shyam Agarwal and Bogdan Vasilescu. 2026. SPECMINE: A Large-Scale Corpus of Spec-Driven Development Artifacts. *arXiv preprint arXiv:2608.25202* (2026).
- [3] Devdatta Akhawe and Adrienne Porter Felt. 2013. Alice in Warningland: A Large-Scale Field Study of Browser Security Warning Effectiveness. In *22nd USENIX Security Symposium (USENIX Security 13)*. 257–272.
- [4] Juan Altmayer Pizzorno and Emery D Berger. 2025. Coverup: Effective high coverage test generation for python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919.
- [5] Mauricio Aniche, Christoph Treude, Igor Steinmacher, Igor Wiese, Gustavo Pinto, Margaret-Anne Storey, and Marco Aurélio Gerosa. 2018. How Modern News Aggregators Help Development Communities Shape and Share Knowledge. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, 499–510. doi:10.1145/3180155.3180180
- [6] Anthropic. 2025. Enabling Claude Code to Work More Autonomously. <https://www.anthropic.com/news/enabling-claude-code-to-work-more-autonomously> Accessed: 18 September 2026.
- [7] Anthropic. 2026. Improving Our Alignment and Security Efforts. <https://www.anthropic.com/news/improving-alignment-security-efforts> Accessed: 18 September 2026.
- [8] Alberto Bacchelli and Christian Bird. 2013. Expectations, Outcomes, and Challenges of Modern Code Review. In *2013 35th International Conference on Software Engineering (ICSE)*. 712–721. doi:10.1109/ICSE.2013.6606617
- [9] Nishant Balepur, Connor Baumler, Valerie Chen, Eunsol Choi, Rachel Rudinger, and Jordan Lee Boyd-Graber. 2026. (Im)Paired Programming: Coding Agents Improve Productivity but Harm Understanding. *arXiv preprint arXiv:2607.26375* (2026).
- [10] Sebastian Baltes, Marc Cheong, and Christoph Treude. 2026. “An Endless Stream of AI Slop”: How Developers Discuss the Burden of AI-Assisted Software Development. *arXiv preprint arXiv:2603.27249* (2026).
- [11] Sebastian Baltes and Paul Ralph. 2022. Sampling in Software Engineering Research: A Critical Review and Guidelines. *Empirical Software Engineering* 27, 4 (2022), 94. doi:10.1007/s10664-021-10072-8
- [12] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *arXiv preprint arXiv:2507.09089* (2025).
- [13] Tingting Bi, Xin Xia, David Lo, John Grundy, Thomas Zimmermann, and Denae Ford. 2022. Accessibility in Software Practice: A Practitioner’s Perspective. *ACM Transactions on Software Engineering and Methodology* 31, 4, Article 66 (2022), 26 pages. doi:10.1145/3503508
- [14] Cristian Bravo-Lillo, Saranga Komanduri, Lorrie Faith Cranor, Robert W. Reeder, Manya Sleeper, Julie Downs, and Stuart Schechter. 2013. Your Attention Please: Designing Security-Decision UIs to Make Genuine Risks Harder to Ignore. In *Proceedings of the Ninth Symposium on Usable Privacy and Security*. 1–12.
- [15] Valerie Chen, Ameet Talwalkar, Robert Brennan, and Graham Neubig. 2026. Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. 1–19.
- [16] Rudrajit Choudhuri, Christian Bird, Carmen Badea, and Anita Sarma. 2026. To Copilot and Beyond: 22 AI Systems Developers Want Built. *arXiv preprint arXiv:2604.07830* (2026).
- [17] Jürgen Cito, Philipp Leitner, Thomas Fritz, and Harald C. Gall. 2015. The Making of Cloud Applications: An Empirical Study on Software Development for the Cloud. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. Association for Computing Machinery, 393–403. doi:10.1145/2786805.2786826
- [18] Anastasia Danilova, Alena Naiakshina, Stefan Horstmann, and Matthew Smith. 2021. Do you Really Code? Designing and Evaluating Screening Questions for Online Surveys with Programmers. In *2021 IEEE/ACM 43rd International*

- Conference on Software Engineering (ICSE)*. 537–548. doi:10.1109/ICSE43902.2021.00057
- [19] Shipi Dhanorkar, Samir Passi, and Mihaela Vorvoreanu. 2026. Human Oversight of Agentic Systems in Practice: Examining the Oversight Work, Challenges, and Heuristics of Developers Using Software Agents. In *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency*. 6438–6465.
 - [20] David Dworken and Oliver Weller-Davies. 2025. Beyond Permission Prompts: Making Claude Code More Secure and Autonomous. <https://www.anthropic.com/engineering/claude-code-sandboxing> Accessed: 18 September 2026.
 - [21] Felipe Ebert, Fernando Castor, Nicole Novielli, and Alexander Serebrenik. 2021. An Exploratory Study on Confusion in Code Reviews. *Empirical Software Engineering* 26, 1, Article 12 (2021). doi:10.1007/s10664-020-09909-5
 - [22] Linda Erlenhov, Francisco Gomes de Oliveira Neto, and Philipp Leitner. 2020. An Empirical Study of Bots in Software Development: Characteristics and Challenges from a Practitioner’s Perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 445–455. doi:10.1145/3368089.3409680
 - [23] Adrienne Porter Felt, Elizabeth Ha, Serge Egelman, Ariel Haney, Erika Chin, and David Wagner. 2012. Android Permissions: User Attention, Comprehension, and Behavior. In *Proceedings of the Eighth Symposium on Usable Privacy and Security*. 1–14.
 - [24] Haoyu Gao, Peerachai Banyongrakkul, Hao Guan, Mansoor Zahedi, and Christoph Treude. 2026. On Autopilot? An Empirical Study of Human–AI Teaming and Review Practices in Open Source. In *Proceedings of the 23rd International Conference on Mining Software Repositories*. 777–781.
 - [25] Haoyu Gao, Jai Lal Lulla, Hong Yi Lin, Sebastian Baltes, Christoph Treude, and Mansoor Zahedi. 2026. From Registry to Repository: How AI Agent Skills Are Written, Adapted, and Maintained. *arXiv preprint arXiv:2607.00911* (2026).
 - [26] GitHub. 2026. Allowing and Denying Tool Use in GitHub Copilot CLI. <https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli/allowing-tools> Accessed: 23 September 2026.
 - [27] GitHub. 2026. Risks and Mitigations for GitHub Copilot Cloud Agent. <https://docs.github.com/en/copilot/concepts/agents/cloud-agent/risks-and-mitigations> Accessed: 21 September 2026.
 - [28] Rashina Hoda. 2022. Socio-Technical Grounded Theory for Software Engineering. *IEEE Transactions on Software Engineering* 48, 10 (2022), 3808–3832. doi:10.1109/TSE.2021.3106280
 - [29] John Hughes. 2026. How We Built Claude Code Auto Mode: A Safer Way to Skip Permissions. <https://www.anthropic.com/engineering/claude-code-auto-mode> Accessed: 18 September 2026.
 - [30] Ranim Khojah, Francisco Gomes de Oliveira Neto, Mazen Mohamad, Julian Frattini, and Philipp Leitner. 2026. Same Scrutiny, More Time: Eye Tracking Insights into Reviewing LLM-Labelled Code. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. arXiv:2606.26505 [cs.SE] doi:10.48550/arXiv.2606.26505
 - [31] Barbara A. Kitchenham and Shari Lawrence Pfleeger. 2002. Principles of Survey Research Part 2: Designing a Survey. *ACM SIGSOFT Software Engineering Notes* 27, 1 (2002), 18–20. doi:10.1145/566493.566495
 - [32] Barbara A. Kitchenham and Shari Lawrence Pfleeger. 2002. Principles of Survey Research Part 4: Questionnaire Evaluation. *ACM SIGSOFT Software Engineering Notes* 27, 3 (2002), 20–23. doi:10.1145/638574.638580
 - [33] Barbara A. Kitchenham and Shari Lawrence Pfleeger. 2002. Principles of Survey Research: Part 5: Populations and Samples. *ACM SIGSOFT Software Engineering Notes* 27, 5 (2002), 17–20. doi:10.1145/571681.571686
 - [34] Arseny Kravchenko, Vadim Liventsev, Innokentii Konstantinov, Ildar Iskhakov, and Matvey Kukuy. 2026. APPA: Recoverable Information-Flow Control for Real-World LLM Agents. arXiv:2607.24625 [cs.CR] <https://arxiv.org/abs/2607.24625>
 - [35] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80.
 - [36] Hong Yi Lin, Chunhua Liu, Haoyu Gao, Patanamon Thongtanunam, and Christoph Treude. 2025. CodeReviewQA: the code review comprehension assessment for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*. 9138–9166.
 - [37] Yvonna S. Lincoln and Egon G. Guba. 1985. *Naturalistic Inquiry*. Sage Publications, Beverly Hills, CA.
 - [38] Adam W. Meade and S. Bartholomew Craig. 2012. Identifying Careless Responses in Survey Data. *Psychological Methods* 17, 3 (2012), 437–455. doi:10.1037/a0028085
 - [39] Alexandra E. Michael and Franziska Roesner. 2026. How Agents Ask for Permission: User Permissions for AI Agents, from Interfaces to Enforcement. *arXiv preprint arXiv:2607.13718* (2026).
 - [40] Emerson Murphy-Hill, Jenna Butler, and Alexandra Savelieva. 2026. Adoption and Impact of Command-Line AI Coding Agents: A Study of Microsoft’s Early 2026 Rollout of Claude Code and GitHub Copilot CLI. *arXiv preprint arXiv:2607.01418* (2026).
 - [41] OpenAI. 2025. Evaluating Chain-of-Thought Monitorability. <https://openai.com/index/evaluating-chain-of-thought-monitorability/> Accessed: 21 September 2026.

- [42] OpenAI. 2026. Running Codex Safely at OpenAI. <https://openai.com/index/running-codex-safely/>. Accessed: 18 September 2026.
- [43] Marc Oriol, Quim Motger, Jordi Marco, and Xavier Franch. 2025. Multi-agent debate strategies to enhance requirements engineering with large language models. In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*. IEEE, 527–534.
- [44] Raja Parasuraman and Victor Riley. 1997. Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors* 39, 2 (1997), 230–253.
- [45] Advait Sarkar and Ian Drosos. 2025. Vibe Coding: Programming Through Conversation with Artificial Intelligence. *arXiv preprint arXiv:2506.23253* (2025).
- [46] Tanjal Shukla, Kevin Feng, Leijie Wang, Mohammad Rostami, and Amy Zhang. 2026. Hedwig: Dynamic Autonomy for Coding Agents Under Local Oversight. In *Proceedings of the ACM Conference on AI and Agentic Systems*. 1293–1299.
- [47] Margaret-Anne Storey, Rashina Hoda, Alessandra Maciel Paz Milani, and Maria Teresa Baldassarre. 2025. Guiding Principles for Mixed Methods Research in Software Engineering. *Empirical Software Engineering* 30, 5, Article 138 (2025). doi:10.1007/s10664-025-10629-x
- [48] Per Erik Strandberg. 2019. Ethical Interviews in Software Engineering. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 1–11. doi:10.1109/ESEM.2019.8870192
- [49] Christoph Treude, Sebastian Baltes, and Marc Cheong. 2026. Operationalizing Ethics for AI Agents: How Developers Encode Values into Repository Context Files. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software*. 106–109.
- [50] Typeform. [n. d.]. Typeform. <https://www.typeform.com/>. Accessed: 10 September 2026.
- [51] Anthony Vance, Jeffrey L. Jenkins, Bonnie Brinton Anderson, Daniel K. Bjornn, and C. Brock Kirwan. 2018. Tuning Out Security Warnings: A Longitudinal Examination of Habituation Through fMRI, Eye Tracking, and Field Experiments. *MIS Quarterly* 42, 2 (2018), 355–380.
- [52] Roberto Verdecchia, Emelie Engström, Patricia Lago, Per Runeson, and Qunying Song. 2023. Threats to Validity in Software Engineering Research: A Critical Reflection. *Information and Software Technology* 164 (2023), 107329. doi:10.1016/j.infsof.2023.107329
- [53] Zhiyuan Wan, Xin Xia, David Lo, and Gail C. Murphy. 2021. How Does Machine Learning Change Software Development Practices? *IEEE Transactions on Software Engineering* 47, 9 (2021), 1857–1871. doi:10.1109/TSE.2019.2937083
- [54] Peiran Wang, Ying Li, and Yuan Tian. 2026. Reframing LLM Agent Security as an Agent-Human Interaction Problem. arXiv:2605.24309 [cs.CR] <https://arxiv.org/abs/2605.24309>
- [55] Mairieli Wessel, Igor Wiese, Igor Steinmacher, and Marco Aurelio Gerosa. 2021. Don’t Disturb Me: Challenges of Interacting with Software Bots on Open Source Software Projects. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW2, Article 301 (2021). doi:10.1145/3476042
- [56] Primal Wijesekera, Arjun Baokar, Ashkan Hosseini, Serge Egelman, David Wagner, and Konstantin Beznosov. 2015. Android Permissions Remystified: A Field Study on Contextual Integrity. In *24th USENIX Security Symposium (USENIX Security 15)*. 499–514.
- [57] Yuhao Wu, Ke Yang, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2026. Towards Automating Data Access Permissions in AI Agents. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2609–2627.
- [58] Marvin Wyrich and Justus Bogner. 2024. Beyond Self-Promotion: How Software Engineering Research Is Discussed on LinkedIn. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society*. ACM, 85–95. doi:10.1145/3639475.3640113
- [59] Ting Yan. 2026. Do User-Authored Permission Policies Improve Protection Against AI Agent Overreach? arXiv:2608.27443 [cs.HC] <https://arxiv.org/abs/2608.27443>
- [60] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [61] Jieyu Zhou, Aryan Roy, Sneh Gupta, Daniel Weitekamp, and Christopher J. MacLellan. 2026. When Should Users Check? Modeling Confirmation Frequency in Multi-Step Agentic AI Tasks. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI ’26)*. Association for Computing Machinery, New York, NY, USA, Article 1649, 20 pages. doi:10.1145/3772318.3790655